UNIVERSIDADE FEDERAL DO CEARÁ

Test Smells em projetos OpenSource

Disciplina: Qualidade de Software
Equipe: Randson da Silva Alves, 568002; Paulo Sergio Vieira de Oliveira, 566589.
Projeto: kennethreitz/responder
Fork: Randson-Silva/responder

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic (Lote de 10 ocorrências refatoradas, abrangendo testes nativos, autenticações e rotas nos arquivos test_auth.py, test_auth_jwt.py, test_cli.py e test_clientgen.py)
Minhas principais dificuldades na remoção do code smell são: Eliminar a lógica imperativa e as estruturas de controle (if, for, while) contidas dentro do escopo dos testes sem alterar os comportamentos originais (que muitas vezes já apresentavam falhas esperadas no repositório). Os desafios principais foram: 1) Remover blocos if usados dentro de simulações (mocks) ou como atalhos para pular testes (skip); 2) Eliminar laços for usados para fazer asserções múltiplas (bulk assertions), o que tornaria os tracebacks de erro confusos; 3) Substituir laços while utilizados como "espera ativa" (polling) de threads assíncronas sem travar o event loop; 4) Purgar variáveis de estado auxiliares que dependiam de repetição para checar ordenação de logs.
Usei o CATSPY para: Definição teórica do problema, consulta de exemplos práticos e validação das estratégias arquiteturais recomendadas para substituir lógicas imperativas de teste por um fluxo linear e declarativo.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Substituição da lógica imperativa por abordagens funcionais, declarativas e lineares. As principais técnicas utilizadas em todo o lote foram: • Loop Unrolling (Desenrolar de Laço): Remoção de estruturas for para a criação de asserções explícitas, individuais e diretas para cada cenário, melhorando a rastreabilidade da falha. • Paradigma Funcional (map/filter com lambdas): Substituição de iterações por filtragens puras (filter) para validar ausência/presença de sub-strings, e uso de map para capturar múltiplos índices de log ou executar chamadas HTTP massivas (flood), concentrando a checagem em um único assert comparando listas (assert status_codes == [401] * n). • Dictionary Lookup: Troca de cadeias condicionais if/else dentro de funções mockadas por um dicionário local, valendo-se do retorno semântico do método .get(). • Extração de Guard Clauses para Decoradores: Movimentação da lógica

condicional de verificação de ambiente (if tool is None) de dentro do teste para os decoradores do Pytest usando pytest.param(marks=pytest.mark.skipif).

- Delegação Assíncrona Nativa: Substituição de laços iterativos while de espera ativa por esperas passivas em thread usando await asyncio.to_thread.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Unknown Test (Lote de 10 ocorrências refatoradas abrangendo autenticações, tokens JWT, CSRF, serialização e depreciações nos arquivos test_auth.py, test_auth_jwt.py, test_csrf_proxy_90.py, test_deprecations_810.py e test_ext_810.py).

Minhas principais dificuldades na remoção do code smell são:
O principal desafio em lidar com esse smell foi contornar os "falsos positivos" estruturais apontados pelo analisador estático (TEMPY). Em frameworks modernos como o Pytest, testes de erros podem ser validados implicitamente usando apenas o gerenciador de contexto with pytest.raises(). A dificuldade foi satisfazer as regras restritas da métrica (que exige a presença estrita da palavra-chave assert) sem poluir o código com validações redundantes, garantindo que a captura de exceções se mantivesse elegante e útil.

Usei o CATSPY para:
Consulta sobre a definição teórica do "Unknown Test" em Python/Pytest e identificação de técnicas de boas práticas para explicitar afirmações sobre exceções capturadas e comportamentos de caminho feliz.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Explicitação de asserções em contextos de exceção e fluxos de criação. Apliquei o padrão de captura em duas frentes:

- Para testes de exceção: Refatorei os blocos with pytest.raises(Error): para capturar a instância do erro usando o alias as exc_info. Em seguida, introduzi a asserção explícita requerida verificando a integridade da classe gerada (assert exc_info.type is Error).

- Para testes de caminho feliz nulos: Nos testes em que instâncias eram criadas apenas para confirmar que o interpretador não falharia, atribuí as chamadas a variáveis locais e inseri asserções lógicas básicas para satisfazer o validador (assert instancia is not None ou validações com any()/all() para lotes).

UNIVERSIDADE
FEDERAL DO CEARÁ

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Exception Handling (Tratamento de Exceções inadequado em testes).
Minhas principais dificuldades na remoção do code smell são: Garantir que a verificação de exceções esperadas e a liberação de recursos ocorram utilizando os recursos nativos do framework de testes (como pytest.raises, fixtures ou monkeypatch), sem alterar o comportamento esperado e a cobertura do teste original, evitando o uso explícito de blocos try/except/finally diretamente no corpo do teste.
Usei o CATSPY para: Definição. O catálogo ajudou a entender que capturar exceções para forçar falhas (pytest.fail) ou para ignorá-las silenciosamente (pass) é um antipadrão. Testes devem delegar o tratamento de falhas ao executor de testes e usar mecanismos próprios para asserts de erro.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Remoção de try/except redundantes (deixando a exceção ser propagada e falhar o teste naturalmente); Substituição de try/except passivos por contexto com pytest.raises; Extração de blocos try/finally (usados para manipulação de estado global como sys.modules) substituindo pelo uso da fixture nativa monkeypatch do Pytest.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Sleepy Test (Uso de pausas explícitas como time.sleep() no corpo ou auxiliares dos testes).
Minhas principais dificuldades na remoção do code smell são: Substituir esperas arbitrárias por sincronização explícita e determinística (utilizando threading.Event, controle de ciclo de vida de tarefas em background e simulação do avanço do tempo via monkeypatch), garantindo que o teste não falhe por condições de corrida (race conditions) nem fique lento.
Usei o CATSPY para: Definição e refatoração. A referência auxiliou na identificação do uso de esperas passivas em testes assíncronos e concorrentes como um antipadrão que afeta o desempenho e a estabilidade da suíte de testes.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Substituição de chamadas de sleep por sincronização baseada em eventos (threading.Event); Avanço de relógio/tempo simulado via monkeypatch para testes com expiração por tempo

(ex: Redis TTL / janelas de Rate Limit);
Remoção de esperas redundantes, delegando a sincronização às fixtures ou métodos de encerramento nativos (ex: join() de threads/processos).